서버프로그램 구현

풀스택 개발자 과정

1조: 박한결, 박영준, 김선길, 원병선

목차

01 구동 확인

02 ProductDTO
김선길

03 ProductRestController
박영준

04 ProductRepository
박영준

05 ProductServiceImpl
박한결

06 ProductServiceTest
박한결

07 ProductBatch
원병선

08 ProductBatchTest
원병선

GitHub: https://github.com/Fullstack-Tests/Server-Program-Implementation

01 구동 확인 - COMMIT

```
5c427ae (HEAD -> develop, origin/develop, origin/HEAD) V1.0 [조장] 최종 확인 완료
a62943c V0.3 Merge pull request #8 from Fullstack-Tests/feature/원병선
4805d87 (origin/feature/원병선) Merge branch 'develop' into feature/원병선
01f55ac V0.2 Merge pull request #7 from Fullstack-Tests/feature/박영준
b1a5c31 V0.1 Merge pull request #6 from Fullstack-Tests/feature/박한결
014e0ce V0.0 Merge pull request #3 from Fullstack-Tests/feature/김선길
9243a36 (origin/feature/박영준) V 0.8 박영준 - 품절(stock=0) 상품 건수 조회용 쿼리메서드를 선언
a62cac3 V 0.7 박영준 - 수정 시 자기 자신(id)을 제외한 상품명 중복 검사용 쿼리메서드를 선언
fa2709a V 0.6 박영준 - 상품명 중복 검사용 쿼리메서드를 선언
b4fbea5 원병선 - 반환된 품절 건수가 0 이상인지 검증
2314954 원병선 - productBatch.checkSoldOut() 호출
927e06f V 0.5 박영준 - 일괄 등록 구현
7e0f6ca V 0.4 박영준 - 삭제 구현
8a4e9e3 V 0.3 박영준 - 수정 구현
ccdd856 V 0.2 박영준 - 등록 구현
38a3344 원병선 - productRepository.countByStock(0) 을 호출하고, 결과(long)를 int 로 변환하여 return
30ce925 V 0.1 박영준 - 단건 조회 구현
32cf6ca V 0.0 박영준 - 목록 조회 구현
f450ac2 품절(stock=0) 상품 건수 조회용 쿼리메서드를 선언(수정)
15800f7 원병선 - 품절(stock=0) 상품 건수 조회용 쿼리메서드를 선언
76d466b 원병선 - 자기 자신(id)을 제외한 상품명 중복 검사용 쿼리메서드 선언
3132bd9 (origin/feature/박한결) V 0.7 박한결 - 일괄등록 중간 실패 시 전체 롤백 테스트 구현
ff6d8b5 원병선 -  상품명 중복 검사용 쿼리메서드를 선언
b4d1e54 V 0.6 박한결 - 정상 등록 테스트 구현
c0b7e33 원병선 - @transactional(readOnly = true) 애너테이션을 붙이기
0591429 V 0.5 박한결 - 일괄 등록(한 트랜잭션) 구현
b2537df V 0.4 박한결 - 목록 조회(페이징) 구현
3bdbd93 V 0.3 박한결 - 단건 조회 구현
7568971 V 0.2 박한결 - 상품 삭제 구현
141cfff V 0.1 박한결 - 상품 수정 구현
25b4590 V 0.0 박한결 - 상품 등록 구현
a24657c (origin/feature/김선길) V0.1 김선길 - Entity -> DTO 변환(static) 메서드 구현
f2a6975 V0.0 김선길 - Entity 변환 메서드 구현
9af7fdb (origin/main) V0.0 [조장] 베이스 업로드
68e1320 Initial commit
```

commit 확인

01 구동 확인 - MySQL

```sql
-- S-Product 상품 관리 스키마 (MySQL testdb / localhost:3306 / root / 1234)
DROP TABLE IF EXISTS tbl_product;

CREATE TABLE tbl_product (
    id        BIGINT       NOT NULL AUTO_INCREMENT,
    name      VARCHAR(100) NOT NULL,
    price     INT          NOT NULL,
    stock     INT          NOT NULL,
    category  VARCHAR(50),
    create_at TIMESTAMP,
    PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- seed 3건
INSERT INTO tbl_product VALUES (null, '기계식 키보드', 89000,  30, 'PC주변기기', now());
INSERT INTO tbl_product VALUES (null, '무선 마우스',   29000, 100, 'PC주변기기', now());
INSERT INTO tbl_product VALUES (null, '27인치 모니터', 245000, 15, '디스플레이', now());

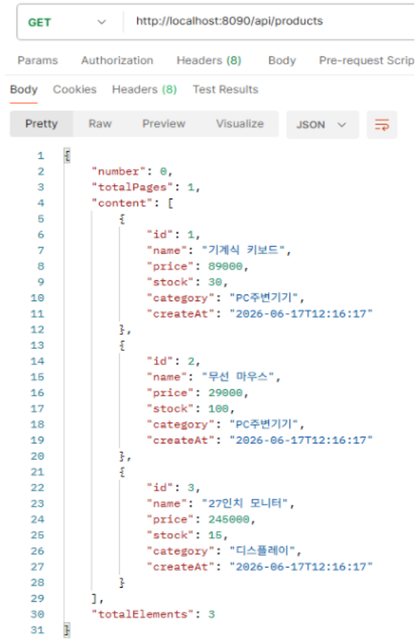
```

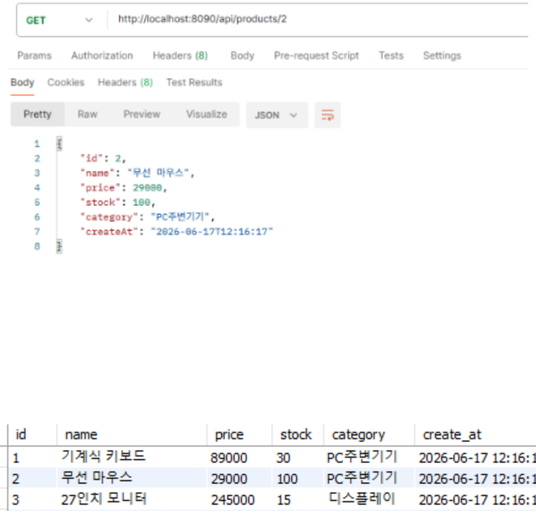
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	name	price	stock	category	create_at
1	기계식 키보드	89000	30	PC주변기기	2026-06-17 12:16:17
2	무선 마우스	29000	100	PC주변기기	2026-06-17 12:16:17
3	27인치 모니터	245000	15	디스플레이	2026-06-17 12:16:17
NULL	NULL	NULL	NULL	NULL	NULL

MySQL 테이블 생성 & 내용 작성

01 구동 확인 - 목록 (전체 + 단건)





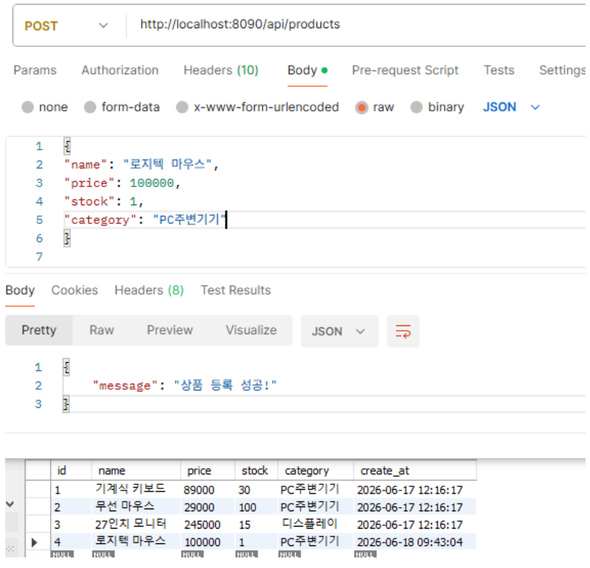
id	name	price	stock	category	create_at
1	기계식 키보드	89000	30	PC주변기기	2026-06-17 12:16:17
2	무선 마우스	29000	100	PC주변기기	2026-06-17 12:16:17
3	27인치 모니터	245000	15	디스플레이	2026-06-17 12:16:17



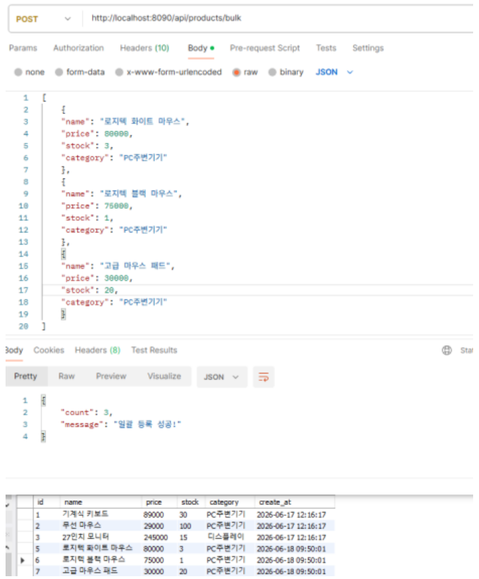
상품 목록 확인 (Postman)

1. GET /api/products 설정 후 send 버튼을 클릭하면
JSON 타입으로 목록 조회가 가능하다.
2. GET /api/products/{id} 설정 후 {id} 값의 번호를 넣어주면
해당 번호에 맞는 상품을 JSON 타입으로 단건 조회가 가능하다.

01 구동 확인 - 등록 (단건 + 일괄)



id	name	price	stock	category	create_at
1	기계식 키보드	89000	30	PC주변기기	2026-06-17 12:16:17
2	무선 마우스	29000	100	PC주변기기	2026-06-17 12:16:17
3	27인치 모니터	245000	15	디스플레이	2026-06-17 12:16:17
4	로지텍 마우스	100000	1	PC주변기기	2026-06-18 09:43:04



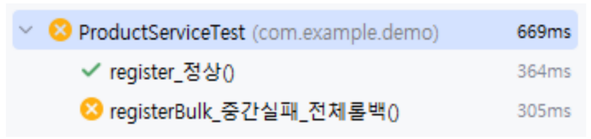
id	name	price	stock	category	create_at
1	기계식 키보드	89000	30	PC주변기기	2026-06-17 12:16:17
2	무선 마우스	29000	100	PC주변기기	2026-06-17 12:16:17
3	27인치 모니터	245000	15	디스플레이	2026-06-17 12:16:17
5	로지텍 화이트 마우스	80000	3	PC주변기기	2026-06-18 09:50:01
6	로지텍 블랙 마우스	75000	1	PC주변기기	2026-06-18 09:50:01
7	고급 마우스 패드	30000	20	PC주변기기	2026-06-18 09:50:01

상품 등록 확인

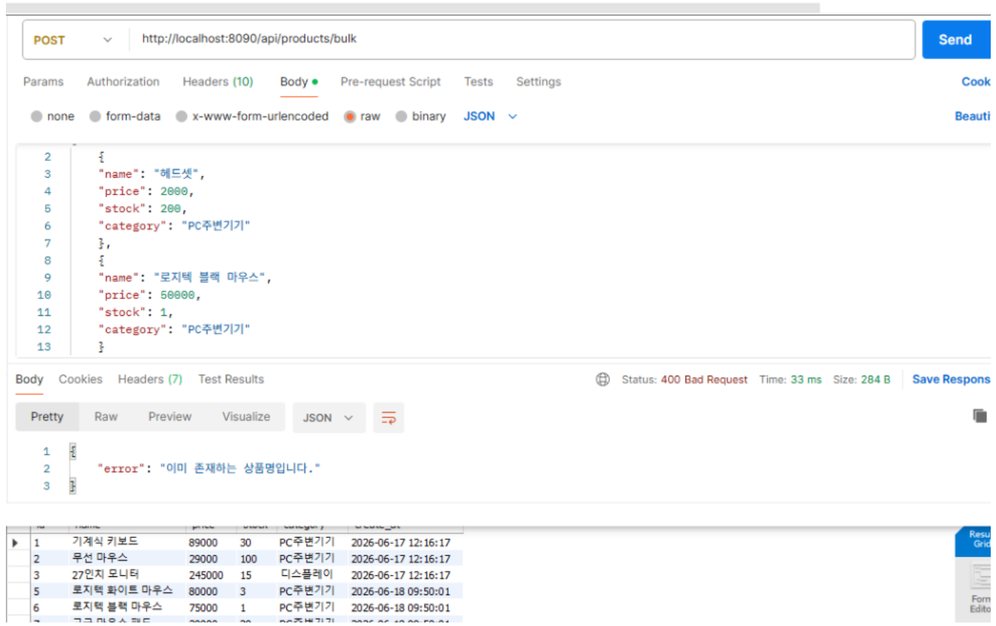
1. POST /api/products 설정 후 Body에 JSON 형태로 상품을 단건 등록할 수 있다.
2. POST /api/products/bulk 설정 후 Body에 대괄호[]로 묶어 여러건 작성하여 JSON 형태로 상품을 일괄 등록할 수 있다.

MySQL에서 확인해보면 상품이 모두 등록된 것을 확인할 수 있다.

01 구동 확인 - 일괄등록 중간실패_전체롤백



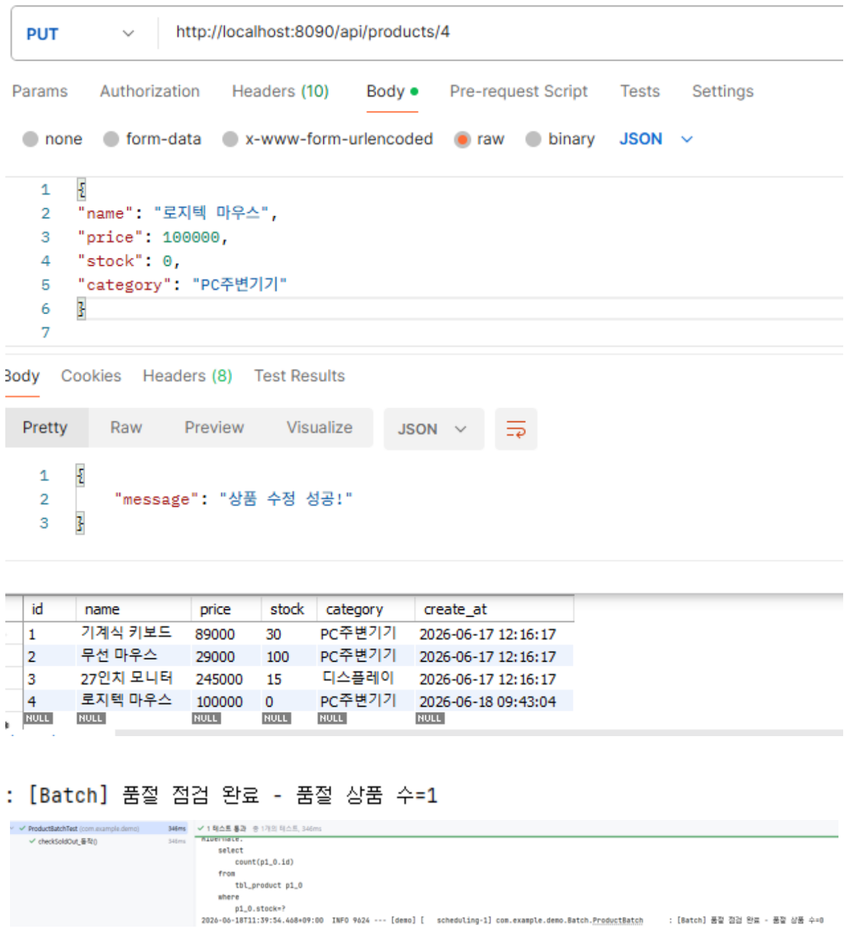

```
ec-1] c.e.d.C.G.GlobalExceptionHandler         : [Biz] 이미 존재하는 상품명입니다.
ec-1] .m.m.a.ExceptionHandlerExceptionResolver : Resolved [com.example.demo.Exception.MyBizException: 이미 존재하는 상품명입니다.]
```



중간실패_전체롤백

POST /api/products/bulk 설정 후 상품을 일괄등록시
서비스 메서드 내부에는 @Transactional이 적용되어 있어, 리스트 중 이미 존재하는 상품명이 포함된 경우 작업을 중단하고 이미 처리된 항목까지 모두 롤백되어 전체 작업이 취소됨.

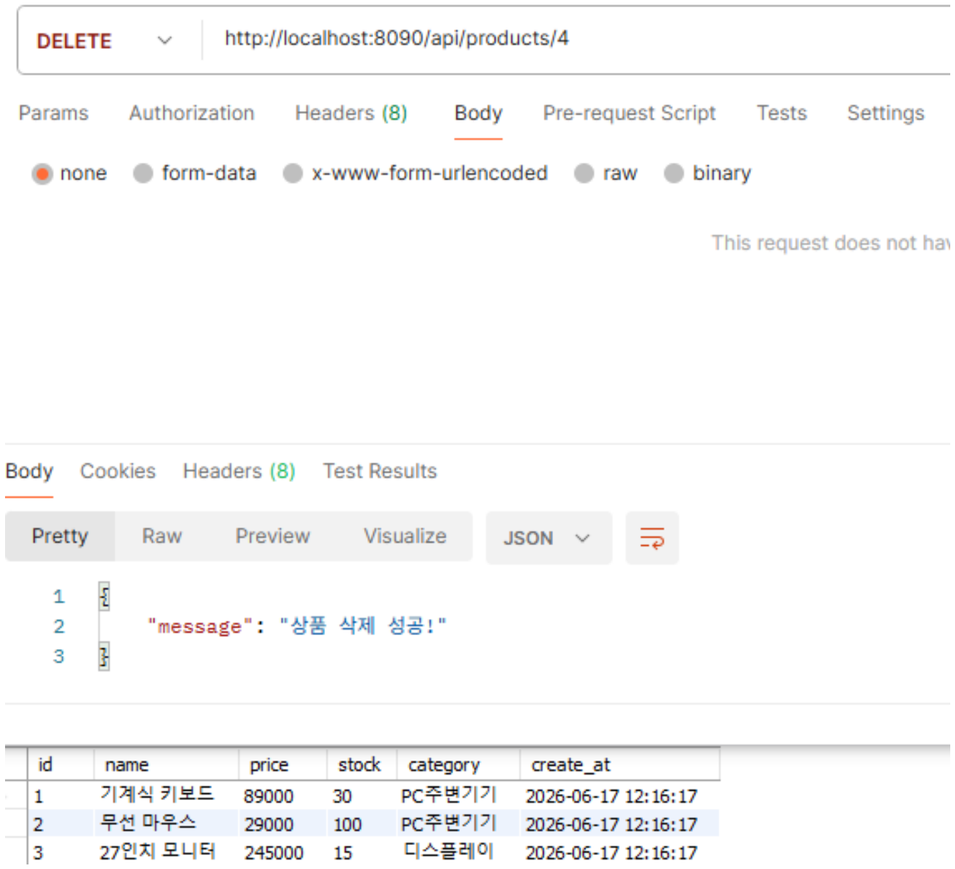
01 구동 확인 - 수정&Batch 구동, 품절 점검 로그



상품 수정 확인

1. PUT /api/products/{id} 설정 후 Body에 JSON 형태로 수정 값을 작성해 상품을 수정할 수 있다.
MySQL에서 확인해보면 상품이 수정된 것을 확인할 수 있다.
2. 상품 개수를 0으로 설정 하면 Batch의 품절 점검이 작동해 @Scheduled 스케줄러를 통해 품절 상품 수가 1개로 늘어난 것을 확인할 수 있다.
3. Batch 테스트를 해보면 정상 구동하는 것을 확인할 수 있다.

01 구동 확인 - 삭제

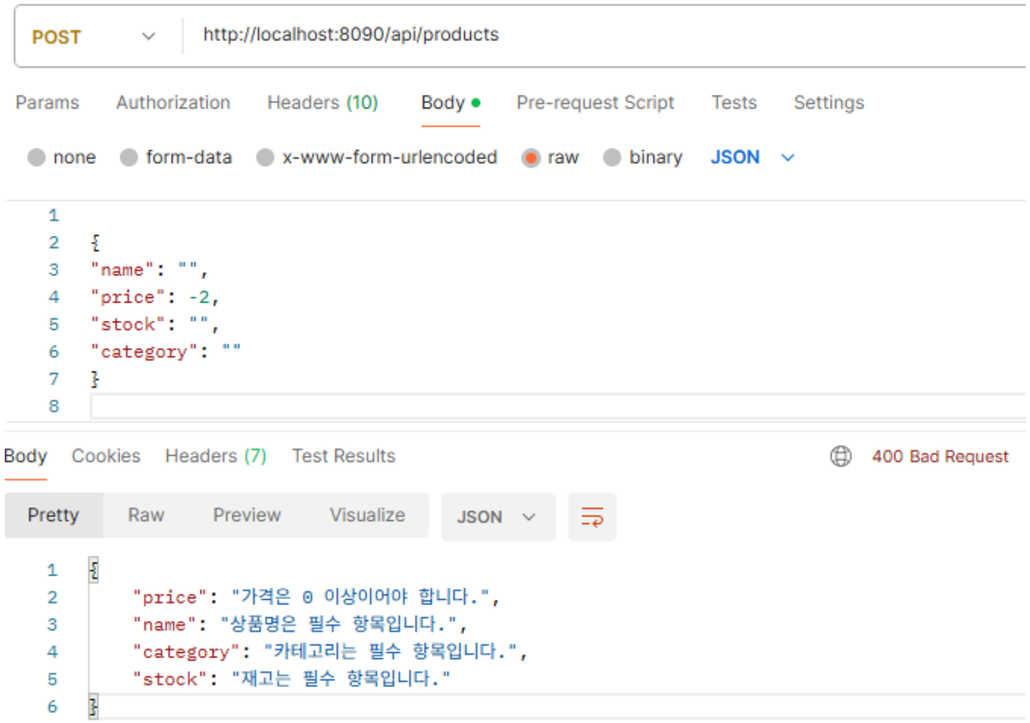


id	name	price	stock	category	create_at
1	기계식 키보드	89000	30	PC주변기기	2026-06-17 12:16:17
2	무선 마우스	29000	100	PC주변기기	2026-06-17 12:16:17
3	27인치 모니터	245000	15	디스플레이	2026-06-17 12:16:17

상품 삭제 확인

1. DELETE /api/products/{id} 설정해 상품을 삭제할 수 있다.
MySQL에서 확인해보면 상품이 삭제된 것을 확인할 수 있다.

01 구동 확인 - 값 검증



@Valid 검증 실패 응답 확인

02 ProductDTO - 김선길

```java
// ===== 기능(메서드) : 학생 구현 =====

// TODO: DTO → Entity 변환 메서드를 구현하라.
//  - Product.builder() 로 id/name/price/stock/category/createAt 를 매핑하여 Product 객체를 반환한다.
public Product toEntity() {
    return Product.builder()
            .id(id)
            .name(name)
            .price(price)
            .stock(stock)
            .category(category)
            .createAt(createAt)
            .build();
}
```

ProductDTO

1. toEntity() 메서드는 DTO 객체의 데이터를 Entity 객체로 변환하는 역할

2. Product.builder()를 사용하여 id, name, price, stock, category, createAt 값을 Product 엔티티에 매핑

3. build()를 호출해 완성된 Product 객체를 생성하여 반환

02 ProductDTO - 김선길

```
    // TODO: Entity → DTO 변환(static) 메서드를 구현하라.
    //  - ProductDTO.builder() 로 Product 의 각 필드를 매핑하여 ProductDTO 객체를 반환한다.
    public static ProductDTO from(Product p) {
        return  ProductDTO.builder()
                .id(p.getId())
                .name(p.getName())
                .price(p.getPrice())
                .stock(p.getStock())
                .category(p.getCategory())
                .createAt(p.getCreateAt())
                .build();
    }
}
```

ProductDTO

1. from() 메서드는 Product 엔티티 객체를 ProductDTO 객체로 변환하는 정적 (static) 메서드

2. ProductDTO.builder()를 사용하여 엔티티의 id, name, price, stock, category, createAt 값을 DTO에 매핑

3. build()를 호출해 완성된 ProductDTO 객체를 생성한 후 반환

03 ProductRestController - 박영준

```java
// TODO: 목록 조회
//  - productService.list(pageable) 호출
//  - 결과의 content / number / totalPages / totalElements 를 Map 에 담아
//    ResponseEntity 로 200(OK) 반환
@GetMapping(produces = MediaType.APPLICATION_JSON_VALUE)  pyj-0313 +1
public ResponseEntity<Map<String, Object>> list(Pageable pageable) {
    log.info("GET /api/products..." + pageable);    //요청 로그
    Map<String, Object> responseMap = new HashMap<>();  //응답 데이터를 담을 Map 생성

    Page<ProductDTO> r = productService.list(pageable); //페이징 처리된 상품 목록 조회

    responseMap.put("content", r.getContent()); //현재 페이지의 데이터 목록
    responseMap.put("number", r.getNumber());   //현재 페이지 번호 (0부터 시작)
    responseMap.put("totalPages", r.getTotalPages());   //전체 페이지 수
    responseMap.put("totalElements", r.getTotalElements()); //전체 데이터 건수

    return ResponseEntity.ok(responseMap);  //200 OK + 응답 데이터 반환
}
```

```java
// TODO: 단건 조회
//  - productService.get(id) 결과를 200(OK) 으로 반환
@GetMapping(value = "/{id}", produces = MediaType.APPLICATION_JSON_VALUE)  1gyeolpark +1
public ResponseEntity<ProductDTO> get(@PathVariable("id") Long id) {

    return ResponseEntity.ok(productService.get(id));   //id로 단건 조회 후 결과를 바로 200 OK로 반환
}
```

ProductRestController 목록 조회 + 단건 조회

- 목록 조회

클라이언트가 보낸 페이지, 사이즈, 정렬 같은 쿼리 파라미터를 Spring이 자동으로 Pageable 객체로 바인딩해서 받고 productService.list(pageable)을 호출하면 Page<ProductDTO> 형태의 결과가 돌아옴. 여기서 실제 데이터 목록 (content)과 현재 페이지 번호(number), 전체 페이지 수(totalPages), 전체 데이터 건수(totalElements)를 꺼내서 Map에 담아 응답으로 보내는 구조

- 단건 조회

URL 경로에 들어있는 id 값을 받아서 productService.get(id)를 호출하고, 그 결과로 나온 ProductDTO 객체를 변수에 따로 담지 않고 바로 200 응답에 실어서 반환. 목록 조회와 달리 페이징 관련 정보 없이 데이터 자체만 그대로 보냄

03 ProductRestController - 박영준

```java
// TODO: 등록
//  - @RequestBody @Valid ProductDTO + BindingResult 로 검증
//  - bindingResult.hasErrors() 이면 FieldError 들을 돌며 필드별 메시지를 Map 에 담아 400(BAD_REQUEST) 반환
//  - 통과 시 productService.register(dto) 호출 후 200 + "상품 등록 성공!" 메시지 반환
@PostMapping(consumes = MediaType.APPLICATION_JSON_VALUE)  pyj-0313 +1
public ResponseEntity<Map<String, Object>> add(@RequestBody @Valid ProductDTO dto,
                                               BindingResult bindingResult) {
    Map<String, Object> responseMap = new HashMap<>();  //응답 데이터를 담을 Map 생성
    if (bindingResult.hasErrors()) {    //검증(@Valid) 실패한 필드가 있는 경우
        for (FieldError error : bindingResult.getFieldErrors()) {   //오류난 필드들을 순회
            log.info("Error Field : " + error.getField() + " Error Message : " + error.getDefaultMessage()); //필드명/에러메시지 로그
            responseMap.put(error.getField(), error.getDefaultMessage());   //필드명을 key, 에러 메시지를 value로 저장
        }
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(responseMap); //400 Bad Request로 검증 오류 응답
    }
    productService.register(dto);   //검증 통과 시 상품 등록 처리

    responseMap.put("message", "상품 등록 성공!");    //성공 메시지 저장
    return ResponseEntity.status(HttpStatus.OK).body(responseMap); //200 OK 응답
}
// TODO: 수정
//  - @PathVariable id + @RequestBody @Valid ProductDTO + BindingResult
//  - 검증 오류 처리는 등록과 동일 / 통과 시 productService.modify(id, dto) → 200 + "상품 수정 성공!"
@PutMapping(value = "/{id}", consumes = MediaType.APPLICATION_JSON_VALUE)  pyj-0313 +1
public ResponseEntity<Map<String, Object>> update(@PathVariable("id") Long id,
                                                  @RequestBody @Valid ProductDTO dto,
                                                  BindingResult bindingResult) {
    Map<String, Object> responseMap = new HashMap<>();  //응답 데이터를 담을 Map 생성
    if(bindingResult.hasErrors()){  //검증(@Valid) 실패한 필드가 있는 경우
        for (FieldError error : bindingResult.getFieldErrors()){    //오류난 필드들을 순회
            log.info("Error Field : " + error.getField() + " Error Message : " + error.getDefaultMessage()); //필드명/에러메시지 로그
            responseMap.put(error.getField(), error.getDefaultMessage());   //필드명을 key, 에러 메시지를 value로 저장
        }
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(responseMap); //400 Bad Request로 검증 오류 응답
    }
    productService.modify(id, dto); //검증 통과 시 id에 해당하는 상품 수정 처리

    responseMap.put("message", "상품 수정 성공!");    //성공 메시지 저장
    return ResponseEntity.status(HttpStatus.OK).body(responseMap); //200 OK 응답
}
```

ProductRestController 등록 + 수정

- 등록

요청 본문으로 전달된 ProductDTO를 @Valid로 검증하고, 그 결과를 BindingResult로 전달받는 구조임. 검증 과정에서 오류가 발생한 필드가 있을 경우 bindingResult.getFieldErrors()를 통해 오류 필드를 순회하며 필드명과 오류 메시지를 Map에 저장한 뒤 400(BAD_REQUEST)으로 응답하고 메서드를 종료.
검증을 통과한 경우에는 productService.register(dto)를 호출하여 실제 등록을 처리하고, 성공 메시지를 MAP에 담아 200(OK)으로 응답

- 수정

등록과 동일한 흐름으로 동작하며, 차이점은 URL 경로를 통해 id 값을 함께 전달받는다는 점과 등록 대신 productService.modify(id,dto)를 호출하여 해당 id의 상품을 수정함. 검증 실패 시 처리 방식과 응답 형식은 등록 기능과 동일하게 구성

03 ProductRestController - 박영준

```java
// TODO: 삭제
//  - productService.remove(id) 호출 후 200 + "상품 삭제 성공!" 메시지 반환
@DeleteMapping("/{id}")  pyj-0313 +1
public ResponseEntity<Map<String, Object>> delete(@PathVariable("id") Long id) {
    Map<String, Object> responseMap = new HashMap<>(); // 응답 데이터를 담을 Map 생성

    productService.remove(id); // id에 해당하는 상품 삭제 처리

    responseMap.put("message", "상품 삭제 성공!"); // 성공 메시지 저장
    return ResponseEntity.status(HttpStatus.OK).body(responseMap); // 200 OK 응답
}

// TODO: 일괄 등록 (한 트랜잭션, 중간 실패 시 전체 롤백)
//  - @RequestBody List<ProductDTO>
//  - productService.registerBulk(list) 호출 후 등록 건수(count) + "일괄 등록 성공!" 반환
@PostMapping(value = "/bulk", consumes = MediaType.APPLICATION_JSON_VALUE)  pyj-0313 +1
public ResponseEntity<Map<String, Object>> addBulk(@RequestBody List<ProductDTO> list) {
    Map<String, Object> responseMap = new HashMap<>(); // 응답 데이터를 담을 Map 생성

    productService.registerBulk(list); // 리스트 일괄 등록 (서비스 내부 @Transactional로 중간 실패 시 전체 롤백 보장)

    responseMap.put("count", list.size());  // 등록 건수 (트랜잭션 보장으로 여기 도달했다면 전체 등록 성공 의미)
    responseMap.put("message", "일괄 등록 성공!"); // 성공 메시지 저장
    return ResponseEntity.status(HttpStatus.OK).body(responseMap); // 200 OK 응답
}
```

ProductRestController 삭제 + 일괄 등록

- 삭제

URL 경로로 전달된 id 값을 받아 productService.remove(id)를 호출함으로써 해당 id의 상품을 삭제 처리. 별도의 검증 절차 없이 삭제를 수행한 뒤 성공 메시지를 Map에 담아 200(OK)으로 응답

- 일괄 등록

요청 본문으로 전달된 ProductDTO 리스트를 받아 productService.registerBulk(list)를 호출. 해당 서비스 메서드 내부에는 @Transactional이 적용되어 있어, 리스트 중 일부 항목 등록에 실패할 경우 이미 처리된 항목까지 모두 롤백되어 전체 작업이 취소. 따라서 예외 없이 해당 호출이 정상적으로 완료되었다면 리스트의 모든 항목이 등록 되었음을 의미하므로, 등록 건수는 요청받은 리스트의 크기(list.size())로 산출함. 이후 등록 건수와 성공 메시지를 Map에 담아 200(OK)로 응답

04 ProductRepository - 박영준

```java
package com.example.demo.Domain.Common.Repository;

import com.example.demo.Domain.Common.Entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository  6개 사용 위치  1gyeolpark +1
public interface ProductRepository extends JpaRepository<Product, Long> {

    // TODO: 등록 시 상품명 중복 검사용 쿼리메서드를 선언하라.
    //       (힌트: boolean existsByName(String name) - 선언만 하면 구현은 자동 생성)
    boolean existsByName(String name); //등록 시 상품명 중복 검사용 쿼리메서드를 선언  0개의 사용위치  pyj-0313
    // TODO: 수정 시 자기 자신(id)을 제외한 상품명 중복 검사용 쿼리메서드를 선언하라.
    //       (힌트: boolean existsByNameAndIdNot(String name, Long id))
    boolean existsByNameAndIdNot(String name, Long id); //수정 시 자기 자신(id)을 제외한 상품명 중복 검사용 쿼리메서드를 선언
    // TODO(배치): 품절(stock=0) 상품 건수 조회용 쿼리메서드를 선언하라.
    //       (힌트: long countByStock(int stock))
    long countByStock(int stock); //품절(stock=0) 상품 건수 조회용 쿼리메서드를 선언  0개의 사용위치  pyj-0313
}
```

ProductRepository

- existsByName(String name)

등록 시 동일한 상품명이 이미 존재하는지 검사하기 위한 메서드

- existsByNameAndIdNot(String name, Long id)

수정 시 자기 자신을 제외한 나머지 상품들 중에 동일한 이름이 존재하는지 검사하기 위한 메서드

(이 두 메서드는 메서드명 자체에 조건이 포함되어 있어 Spring Data JPA 가 메서드 이름을 분석하여 쿼리를 자동으로 생성하므로, 별도의 구현 코드 없이 선언만으로 동작)

- countByStock(int stock)

재고(stock) 값이 특정 수치, 즉 0인 상품의 건수를 조회하기 위한 메서드로, 품절 상품 현황을 파악하는 배치 작업에서 활용

05 ProductServiceImpl - 박한결

```java
// 상품 등록
@Override  3개 사용 위치
@Transactional // 해당 메서드 실행 중 발생하는 DB 작업을 하나의 트랜잭션으로 처리
public ProductDTO register(ProductDTO dto) {

    if (productRepository.existsByName(dto.getName())) {                // 상품명 중복 검사. 동일한 상품명이 DB에 존재하는지 확인
        throw new MyBizException("이미 존재하는 상품명입니다.");          // 상품명이 중복 존재하면 등록하지 않고 예외 발생시킴
    }
    dto.setCreateAt(LocalDateTime.now());                               // 등록 시간을 현재 시간으로 설정
    Product product = productRepository.save(dto.toEntity());           // DTO → Entity 로 변환해 db에 저장

    return ProductDTO.from(product);                                    // 저장된 Entity → DTO 형태로 변환해 반환
}

// 상품 수정
@Override  1개 사용 위치
@Transactional
public ProductDTO modify(Long id, ProductDTO dto) {

    Product product = productRepository
            .findById(id)                                   // id로 수정할 상품 Entity 조회
            .orElseThrow(() -> new MyBizException("해당 상품을 찾을 수 없습니다.")); // id를 찾을 수 없으면 수정하지 않고 예외 발생시킴

    if (productRepository.existsByNameAndIdNot(dto.getName(), id)) { // 현재 id 제외 동일한 상품명이 DB에 존재하는지 상품명 중복 검사
        throw new MyBizException("이미 존재하는 상품명입니다.");        // 상품명이 중복 존재하면 등록하지 않고 예외 발생시킴
    }

    // Entity 필드 값을 조회해 현재 dto의 값으로 변경
    product.setName(dto.getName());
    product.setPrice(dto.getPrice());
    product.setStock(dto.getStock());
    product.setCategory(dto.getCategory());

    return ProductDTO.from(product);    // 변경된 Entity → DTO 형태로 변환해 반환
}
```

상품 등록 / 수정

상품 등록: 상품명 중복을 확인한 뒤 상품 정보를 DB에 저장하는 기능

@Transactional : 등록 과정 전체를 하나의 트랜잭션으로 처리

existsByName() : 상품명 중복 검사. 중복 시 MyBizException 발생

createAt에 현재 시간 저장해 등록 시간 설정

DTO → Entity 변환 후 DB 저장

저장된 데이터를 Entity → DTO로 변환하여 반환

상품 수정: 기존 상품을 조회한 후 중복을 검사하고 정보를 수정하는 기능

findById() : 수정할 상품 ID 조회. 없으면 MyBizException 발생

existsByNameAndIdNot() : 자기 자신을 제외한 상품명 중복 검사

조회한 Entity의 현재 값을 수정할 DTO 값으로 변경

수정된 데이터를 Entity → DTO로 변환하여 반환

05 ProductServiceImpl - 박한결

```java
// 상품 삭제
@Override  1개 사용 위치
@Transactional
public void remove(Long id) {

    if (!productRepository.existsById(id)) {                    // 삭제할 상품 id 조회
        throw new MyBizException("해당 상품을 찾을 수 없습니다.");  // id를 찾을 수 없으면 삭제하지 않고 예외 발생시킴
    }

    productRepository.deleteById(id);                           // 해당 id 상품을 DB에서 삭제
}

// 단건 조회
@Override  1개 사용 위치
@Transactional(readOnly = true)     // 읽기 전용 트랜잭션 true, 성능 최적화
public ProductDTO get(Long id) {

    Product product = productRepository
            .findById(id)                                       // id로 조회할 상품 Entity 조회
            .orElseThrow(() -> new MyBizException("해당 상품을 찾을 수 없습니다."));

    return ProductDTO.from(product);                            // 조회한 Entity → DTO 형태로 변환해 반환
}
```

상품 삭제 / 단건 조회

상품 삭제: 상품 ID를 기준으로 해당 상품을 DB에서 삭제하는 기능
@Transactional : 삭제 과정을 하나의 트랜잭션으로 처리
existsById() : 해당 상품 ID가 존재하는지 확인, 없으면 MyBizException 발생
deleteById() : 해당 ID의 상품을 데이터베이스에서 삭제

단건 조회: 상품 ID를 기준으로 하나의 상품 정보를 조회하는 기능
@Transactional(readOnly = true) : 조회 전용 트랜잭션으로 성능 최적화
findById() : 상품 ID로 데이터 조회, 없으면 MyBizException 발생
조회된 Entity를 ProductDTO.from()으로 DTO로 변환하여 반환

05 ProductServiceImpl - 박한결

```java
// 목록 조회(페이징)
@Override
@Transactional(readOnly = true)
public Page<ProductDTO> list(Pageable pageable) {

    return productRepository
            .findAll(pageable)          // 페이징 조건에 맞게 상품 목록 조회
            .map(ProductDTO::from);     // 조회한 Entity 목록 DTO 목록으로 변환
}

// 일괄 등록(한 트랜잭션)
@Override  2개 사용 위치
@Transactional
public int registerBulk(List<ProductDTO> list) {

    for (ProductDTO dto : list) {                               // 전달 받은 list를 순회
        if (productRepository.existsByName(dto.getName())) {    // 동일한 상품명이 DB에 존재하는지 상품명 중복 검사
            throw new MyBizException("이미 존재하는 상품명입니다.");// 상품명이 중복 존재하면 예외 발생시켜 트랜잭션 전체 롤백
        }
        dto.setCreateAt(LocalDateTime.now());                   // 등록 시간을 현재 시간으로 설정
        productRepository.save(dto.toEntity());                 // DTO → Entity 로 변환해 db에 저장
    }
    return list.size();     // 등록된 상품 건수 반환
}
```

목록 조회 / 일괄 등록

목록 조회: 상품 목록을 페이징 처리하여 조회하는 기능

@Transactional(readOnly = true) : 조회 전용 트랜잭션으로 성능 최적화

findAll(pageable) : 페이징 조건에 맞게 상품 목록 조회

조회된 Entity 목록을 map(ProductDTO::from)을 통해 DTO 목록 변환해 반환

일괄 등록: 여러 개의 상품을 한 번의 트랜잭션으로 등록하는 기능

@Transactional : 전체 작업을 하나의 트랜잭션으로 처리 (중간 실패 시 전체 롤백)

for (ProductDTO dto : list) {} : 리스트를 순회하며 하나씩 상품 처리

existsByName() : 상품명 중복 검사, 중복 시 MyBizException 발생

createAt에 현재 시간 저장해 등록 시간 설정

DTO → Entity 변환 후 DB 저장

모든 상품이 정상 등록되면 등록된 개수(list.size()) 반환

06 ProductServiceTest - 박한결

```java
// 정상 등록 테스트
@Test
void register_정상() {

    ProductDTO dto = ProductDTO.builder()    // 등록할 상품의 DTO 생성
            .name("케이크")
            .price(50000)
            .stock(6)
            .category("제과")
            .build();

    ProductDTO result = productService.register(dto);  // DB에 상품 등록

    assertNotNull(result.getId());          // DB 저장 시 id 자동 생성되므로 null 아닌지 확인
}

// 일괄등록 중간 실패 시 전체 롤백 테스트
@Test
void registerBulk_중간실패_전체롤백() {

    ProductDTO dto = ProductDTO.builder().name("바게트").price(6000).stock(10).category("제빵").build();  // 중복 유발 상품 1건 DTO 생성
    productService.register(dto);       // DB에 상품 등록 (count = 1)

    List<ProductDTO> dtoList = List.of(         // 등록할 상품 DTO list로 생성
            ProductDTO.builder().name("롤빵").price(15000).stock(7).category("제과").build(),       // 정상 상품 생성
            ProductDTO.builder().name("바게트").price(12000).stock(1).category("제빵").build()       // 이름 중복 상품 DTO 작성, 예외 발생
    );

    assertThrows(MyBizException.class, () -> productService.registerBulk(dtoList));     // 2번째 상품 이름 중복 예외 발생 확인, 트랜잭션 전체 롤백

    long count = productRepository.count();     // DB에 저장된 전체 상품 개수 확인
    assertEquals( expected: 1, count);          // dtoList 전체 롤백되어 현재 첫 dto 등록 1건만 존재(count = 1)
}
```

정상 등록 / 일괄 등록 실패 전체 롤백

정상 등록: 상품이 정상적으로 등록되는지 확인하는 테스트
ProductDTO.builder()를 사용해 테스트 상품 생성
productService.register()를 호출하여 DB에 저장
저장된 결과의 id가 생성되었는지 assertNotNull()로 검증

중간실패_전체롤백: 일괄 등록 중 하나라도 실패하면 전체 롤백되는지 확인하는 테스트
먼저 중복 검증을 위해 상품 1건을 DB에 미리 등록
이후 registerBulk()에 정상 상품 + 중복 상품을 함께 전달
중복 발생 시 MyBizException이 발생하는지 assertThrows()로 검증
@Transactional에 의해 전체 트랜잭션이 롤백되는지 확인
최종적으로 productRepository.count()가 1인지 검증하여 부분 저장이 없는지 확인

07 ProductBatch - 원병선

```java
@Component
@Slf4j
public class ProductBatch {

    @Autowired
    private ProductRepository productRepository;

    @Scheduled(fixedDelayString = "${batch.soldout.delay:60000}")
    // 스케줄러 기능을 통해 "${batch.soldout.delay:60000}" 값을 읽어오고
    // 해당 설정 값이 없으면 기본 값으로 1분을 사용

    public void scheduleSoldOutCheck() {
        int count = checkSoldOut();
        // 메서드를 호출하여 '품절 처리 된 상품의 개수'를 변수 count에 저장
        log.info("[Batch] 품절 점검 완료 - 품절 상품 수=" + count);
        // 콘솔이나 로그 파일에 작업결과를 기록
    }
```

ProductBatch

- ProductBatch 라는 공개 클래스 정의
- @Autowired를 통해 데이터 베이스에 접근 가능
- @Scheduled 기능을 통해 주기적으로 메서드를 실행하도록 설정
- fixedDelayString을 사용해 이전 작업이 끝난 시점을 기준으로 대기시간을 가짐
- application.properties 해당 설정 값이 없으면 기본 값으로 1분을 사용
- 품절 여부를 체크하는 메서드(checkSoldOut)을 호출하고 결과로 반환된 것을 count에 저장
- 로깅 라이브러리를 사용해 콘솔이나 로그 파일에 결과를 기록

07 ProductBatch - 원병선

```
    @Transactional(readOnly = true)  2개 사용 위치
    // 데이터베이스 트랜잭션을 시작하나, '읽기 전용'으로 설정
    // 데이터에 안전하게 접근하고 조회하기 위함
    public int checkSoldOut() {
        return (int) productRepository.countByStock(0);
        // productRepository를 사용해 재고가 0인 상품의 건수를 조회
        // long 타입으로 반환된 것을 int로 변환
        // 재고가 0인 상품의 총 개수를 반환
    }
}
```

ProductBatch

- 데이터베이스 트랜잭션을 시작하지만, '읽기전용(readOnly = true)' 모드로 설정
- 데이터에 안전하게 접근하고 조회하기 위함
- productRepository를 사용해 재고가 0인 상품을 데이터베이스에서 조회
- countByStock(0) 메서드는 long 타입을 반환하므로, int 타입으로 형변환을 시킴
- 재고가 0개(품절)인 상품의 총 개수를 메서드 바깥으로 반환

08 ProductBatchTest - 원병선

```java
@SpringBootTest
class ProductBatchTest {

    @Autowired
    private ProductBatch productBatch;

    // TODO: 배치 동작 테스트 (NCS 4.3)
    //  - productBatch.checkSoldOut() 호출
    //  - 반환된 품절 건수가 0 이상인지 검증(assertTrue(count >= 0))
    @Test
    void checkSoldOut_동작() {
        int count =  productBatch.checkSoldOut();
        // productBatch 클래스의 checkSoldOut()메서드를 호출
        // 품절된 상품의 개수를 int 타입으로 count 변수에 저장
        assertTrue( condition: count >= 0);
        // 검증 메서드로 괄호 안의 조건이 '참'인지 확인
    }
}
```

ProductBatchTest

- @Test 어노테이션을 통해 '테스트 케이스'임을 표시
- 이전에 구현했던 productBatch 클래스의 checkSoldOut() 메서드를 실제로 호출하고 결과를 int 타입의 count 변수에 저장
- 괄호안의 조건이 '참(true)'인지 확인하고, 상품의 개수는 음수가 될 수 없으므로 조회된 count가 이상인지 검증